



C Programming Fundamentals

Program structure, data types, operators, decisions, loops, functions, and engineering practice

Embedded Bare-Metal Diploma

DAY 1

6 Hours

```
#include <stdio.h>

int main(void)
{
    return 0;
}
```

Prepared by Mahmoud Morsy

NTI | National Telecommunication Institute



One session, one complete programming foundation

Session roadmap

00:00**01**

**Program
structure
and build
chain**

01:10**02**

**Data, input
and operators**

02:20**03**

**Decisions
and switch**

03:15**04**

**Loops and
execution
tracing**

04:15**05**

**Functions,
applications
and
assessment**

Teaching rhythm: explain → demonstrate → 5–10 minute task → review



By the end, students can write and trace small C programs

Learning outcomes

- Explain how source code becomes a program.
- Select variables and data types for engineering values.
- Use input, output, arithmetic, relational and logical operators.
- Create decisions with if, else if, else and switch.
- Choose and trace for, while and do-while loops.
- Build reusable functions with parameters and return values.

TODAY'S OUTPUT

8

**working programs
and completed
quizzes**



C gives direct, predictable control over embedded hardware

Why C matters

01

Close to hardware

Addresses, bytes and bits map naturally to registers and memory.

02

Portable structure

The same language supports console practice now and AVR firmware later.

03

Efficient execution

Compiled code can run with limited memory and processing power.

Bare-metal mindset: the program starts, configures hardware, then remains active in an infinite loop.



Every C program begins with a clear entry point

Program structure

```
#include <stdio.h>

int main(void)
{
    printf("Welcome to Embedded C\n");
    return 0;
}
```

READ IT TOP TO BOTTOM

- **Header:** declares library features.
- **main():** the program entry point.
- **Statements:** end with a semicolon.
- **Braces:** group the function body.
- **return 0:** signals successful completion.

C is case-sensitive



The build chain turns C text into machine instructions

Build process



AVR workflow

The ELF keeps rich build information; the HEX carries program bytes for flashing the microcontroller.



A data type tells C how to interpret stored bits

Variables and data types

Type	Engineering use	Example
char	Character or small integer	'R'
int	Whole-number count	1500
float	Measured decimal value	36.7f
double	Higher-precision decimal	3.14159
const	Value that must not change	const float VREF = 5.0f;

Important: exact type sizes can depend on the compiler and target. Embedded projects later use `uint8_t` and `uint16_t` for explicit widths.



Exercise: choose the smallest clear representation

7-minute practice

- 1 Number of motor revolutions
- 2 Temperature of 38.6 °C
- 3 Direction represented by F or R
- 4 Constant supply voltage of 12 V

Work individually • 7 minutes



Answer: the type should match the value and its purpose

Instructor review

```
int revolutions;  
float temperature;  
char direction;  
const int supplyVoltage = 12;
```

ASK WHY

**Why is
temperature
not an int?**

**Why should
supply
voltage be const?**



Input stores a value; output makes program state visible

Input and output

```
int speed;  
  
printf("Enter speed: ");  
scanf("%d", &speed);  
printf("Speed = %d RPM\n", speed);
```

FORMAT SPECIFIERS

Code	Meaning
%d	int
%f	float input
%c	char
%.2f	two decimal places

The & operator gives scanf() the variable's address.



Operators transform values and test engineering conditions

Operators

Category	Operators	Typical question
Arithmetic	+ - * / %	What is the result?
Assignment	= += -= *= /=	How does the value change?
Relational	== != > < >= <=	Is the comparison true?
Logical	&& !	Are combined conditions true?
Increment	++ --	Move one step up or down

Common error: = assigns a value, while == compares two values.



Exercise: calculate voltage from current and resistance

8-minute practice

$$V = I \times R$$

- Read resistance and current as decimal values.
- Calculate voltage using Ohm's law.
- Print the result with two decimal places.

Example: $R = 10 \, \Omega$, $I = 1.5 \, A \rightarrow V = 15.00$
V



Answer: separate input, calculation and output

Instructor review

```
#include <stdio.h>

int main(void)
{
    float resistance, current, voltage;

    scanf("%f", &resistance);
    scanf("%f", &current);
    voltage = current * resistance;

    printf("Voltage = %.2f V\n", voltage);
    return 0;
}
```

INPUT

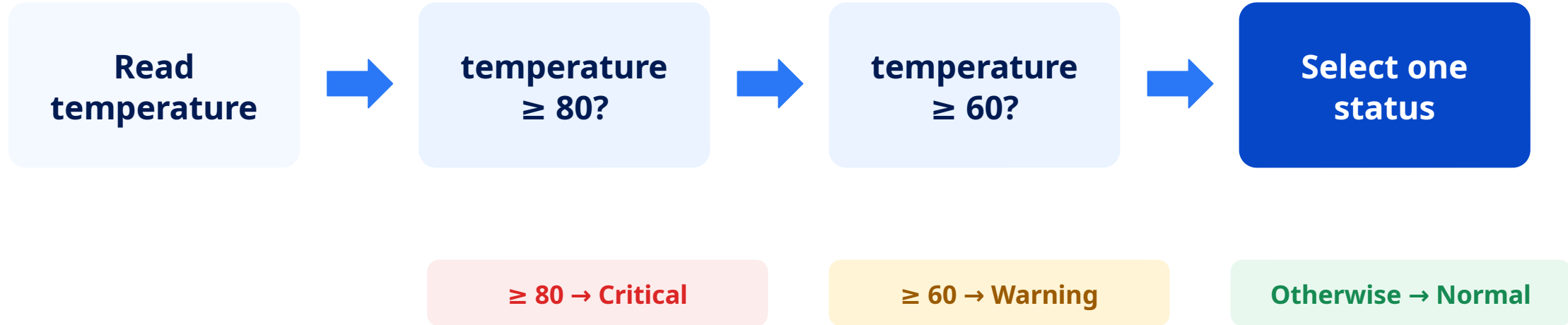
PROCESS

OUTPUT



A decision selects one path from measured conditions

if / else if / else



Order matters: test the highest threshold first.



The first true branch runs; the remaining branches are skipped

Condition example

```
if (temperature >= 80.0f)
{
    printf("Critical\n");
}
else if (temperature >= 60.0f)
{
    printf("Warning\n");
}
else
{
    printf("Normal\n");
}
```

TRACE THREE INPUTS

55 °C **Normal**

65 °C **Warning**

85 °C **Critical**



Exercise: classify a 12 V battery

8-minute practice

Voltage range	Display
Below 10.5 V	Low
10.5 V to below 12.6 V	Normal
12.6 V or higher	Fully Charged

REQUIREMENTS

Read one float

Use if / else if / else

Print one status

Work individually • 8 minutes



Answer: ordered tests define exclusive battery ranges

Instructor review

```
float voltage;  
scanf("%f", &voltage);  
  
if (voltage < 10.5f)  
{  
    printf("Low\n");  
}  
else if (voltage < 12.6f)  
{  
    printf("Normal\n");  
}  
else  
{  
    printf("Fully Charged\n");  
}
```

BOUNDARY CHECK

10.49 → Low
10.50 → Normal
12.59 → Normal
**12.60 → Fully
Charged**



Use switch when one value selects an exact command

switch-case

```
switch (command)
{
    case 1:
        printf("Start\n");
        break;

    case 2:
        printf("Stop\n");
        break;

    case 3:
        printf("Reverse\n");
        break;

    default:
        printf("Invalid\n");
}
```

WHEN TO USE IT

- One integer or character expression
- Several exact constant choices
- A clear default for invalid input
- break prevents fall-through

Ranges usually belong in if / else if.



Choose the loop from the repetition rule

Loops

for

Known count

Take exactly 10 samples

while

Condition first

Repeat while a switch is ON

do-while

Run once first

Show a menu before
checking

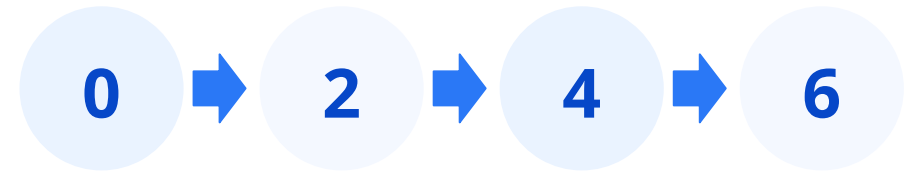
Embedded programs often use while (1) as the main application loop.



Trace the update to predict every loop output

5-minute quiz

```
for (int i = 0; i < 8; i += 2)
{
    printf("%d ", i);
}
```



Answer: 4 executions → 0 2 4 6

How many executions? What is printed?



Exercise: sum the integers from 1 to 10

8-minute practice

$$1 + 2 + 3 + \dots + 10 = ?$$

- Start sum at zero.
- Use a for loop from 1 through 10.
- Accumulate each number into sum.
- Print the final result once.

Do not print inside the loop.

Work individually • 8 minutes



Answer: the accumulator preserves the growing total

Instructor review

```
int sum = 0;

for (int number = 1; number <= 10; number++)
{
    sum += number;
}

printf("Sum = %d\n", sum);
```

TRACE

**0 → 1 → 3 → 6 → 10
→ 15 → ... → 55**

Final result = 55



Functions turn repeated calculations into reusable building blocks

Functions and scope

```
float calculatePower(float voltage, float current)
{
    return voltage * current;
}

int main(void)
{
    float result = calculatePower(12.0f, 1.5f);
    printf("Power = %.2f W\n", result);
    return 0;
}
```

Return type

float

Name

calculatePower

Parameters

voltage, current

Return value

voltage × current

Local variables exist only
inside their block.



Exercise: return the larger of two integers

10-minute practice

```
int largerValue(int first, int second);
```

- Compare the two parameters.
- Return the larger value.
- Call the function from main().
- Print the result.

Test case: `largerValue(25, 18)` → 25

Maximum time • 10 minutes



Answer: one function should have one clear responsibility

Instructor review

```
int largerValue(int first, int second)
{
    if (first > second)
    {
        return first;
    }

    return second;
}

int main(void)
{
    printf("%d\n", largerValue(25, 18));
    return 0;
}
```

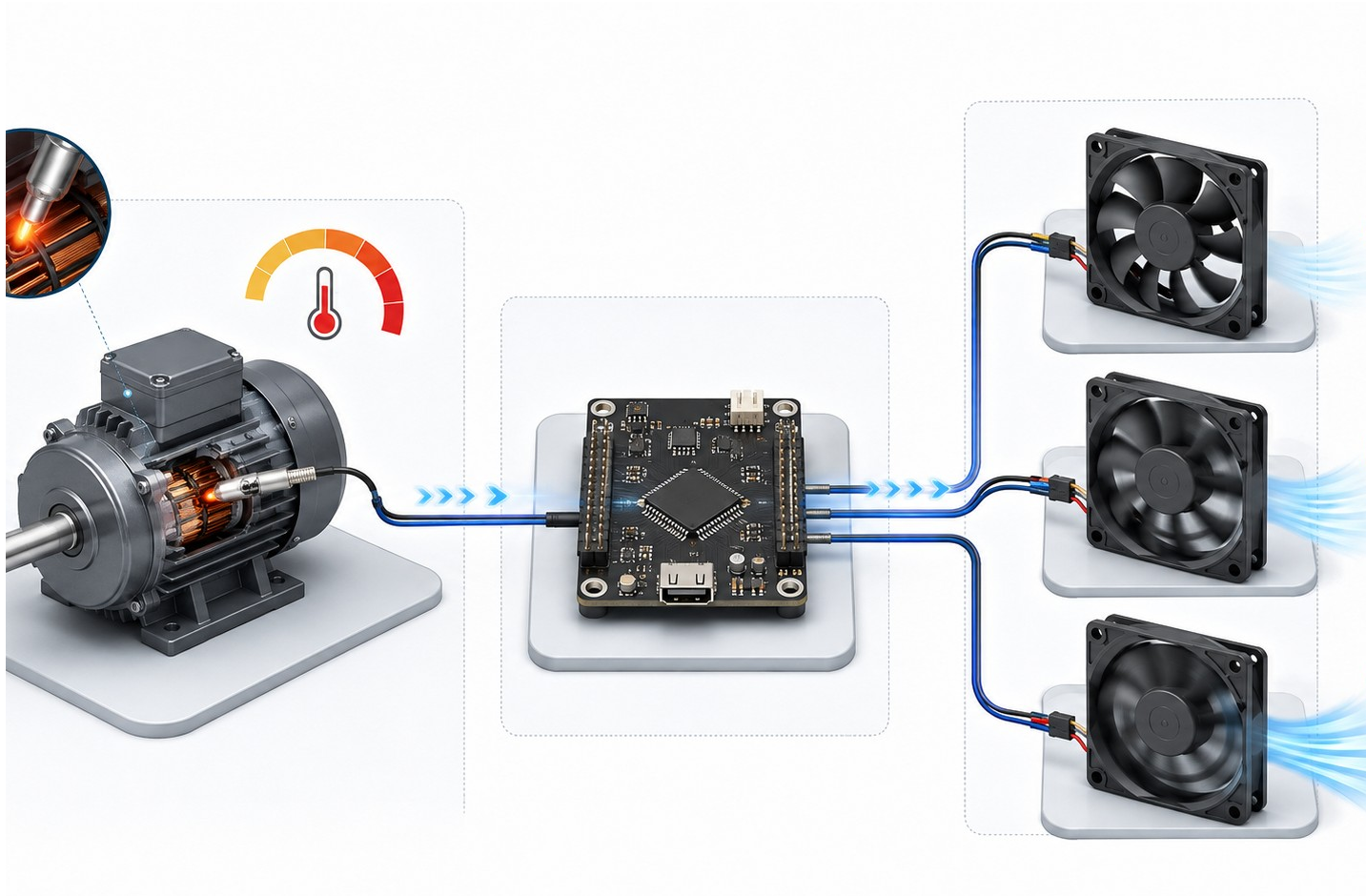
25

returned value



Application: temperature selects the cooling-fan command

W3Schools-style application



getFanSpeed()

Temperature	Return
Below 40 °C	0%
40 to below 70 °C	50%
70 °C or higher	100%

Student working time • 10 minutes



Solution: isolate the decision inside getFanSpeed()

Application review

```
int getFanSpeed(float temperature)
{
    if (temperature >= 70.0f)
        return 100;
    else if (temperature >= 40.0f)
        return 50;
    else
        return 0;
}
```

```
int main(void)
{
    float temperature;
    scanf("%f", &temperature);

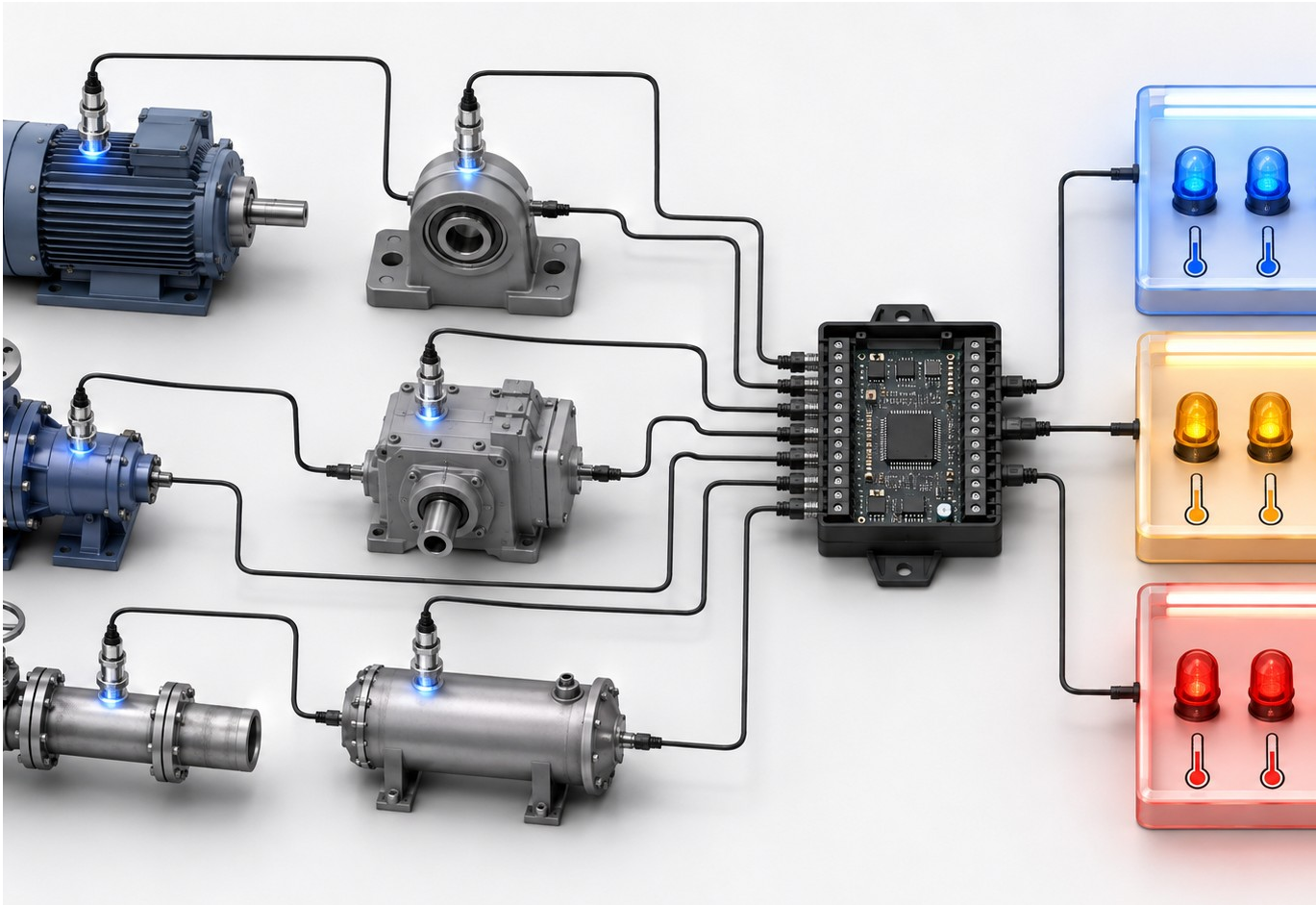
    int speed =
    getFanSpeed(temperature);
    printf("Fan speed = %d%%\n", speed);
    return 0;
}
```

TESTS 30 → 0% 55 → 50% 85 → 100%



Challenge: classify a sequence of sensor readings

HackerRank-style application



INPUT

6
45 62 81 59 75 90

RULES

Safe: below 60

Warning: 60 to below 80

Critical: 80 or higher

Maximum time • 10 minutes



Solution: one loop updates three independent counters

Application review

```
int n, temperature;
int safe = 0, warning = 0, critical = 0;

scanf("%d", &n);

for (int i = 0; i < n; i++)
{
    scanf("%d", &temperature);

    if (temperature >= 80)
        critical++;
    else if (temperature >= 60)
        warning++;
    else
        safe++;
}
```

```
printf("Safe: %d\n",
safe);
printf("Warning: %d\n",
warning);
printf("Critical: %d\n",
critical);
```

Sample output

Safe: 2
Warning: 2
Critical: 2



Final quiz — fundamentals

10-minute assessment • Part 1

- 1 Which function is the C entry point?
A start() B main() C run() D init()
- 2 Complete: ____ temperature = 25.7f;
- 3 Which operator tests equality?
A = B == C != D >=
- 4 Complete: scanf("%d", ____ number);



Final quiz — control flow and functions

10-minute assessment • Part 2

5 Which loop is best for exactly ten repetitions?

6 What does the % operator calculate?

7 Complete: `int add(int a, int b) { ____ a + b; }`

8 If `value = 5` and `value > 3`, does the program print ON or OFF?



Quiz answers

Instructor review

1 B — main()

2 float

3 B — ==

4 &

5 for

6 integer remainder

7 return

8 ON

Use errors to decide what must be revised before the next C session.



The same patterns will control ATmega32 peripherals

Session synthesis

01

Data

stores sensor readings
and commands

02

Decisions

select actions from
thresholds

03

Loops

repeat sampling and
control

04

Functions

become reusable
hardware drivers

Next: arrays, strings, pointers, structures and modular C files



Teaching references

Sources

CORE COURSE MATERIAL

MIT OpenCourseWare — Practical
Programming in C

GNU GCC — Overall Options and build stages

PRACTICE STRUCTURE

W3Schools — C examples, input, conditions
and loops

HackerRank — C practice domain

All applications, quizzes, code solutions and explanatory illustrations in this deck are original course materials. Platform references guided the learning sequence and practice style; no proprietary exercise was copied.

Links are available in the PowerPoint speaker notes.